

1. Introduction

This report documents the design, development, and implementation of a Library Management System built in Java. The project simulates a real-world library system where users can browse, borrow, and return items, and administrators can supervise the entire operation.

The main goal was not just to build something that works using the object-oriented principles we covered throughout the course, including inheritance, polymorphism, abstract classes, interfaces. The project also incorporates exception handling, file I/O with CSV files, recursion, streams, and a variety of Java collections.

2. System Overview

The system is organized with three main ways: managing users, managing library items, and handling the interactions between the two.

2.1 Users

The system supports three types of users, all extending an abstract User class. Students can borrow up to 5 books and books only. Teachers can borrow up to 10 items of any type. Admins do not borrow items but can. However, they have to manage the system, generate reports, and trigger backups.

```
@Getter 9 usages 3 inheritors SavulescuD
@ToString
@EqualsAndHashCode
public abstract class User implements Comparable<User> {
    protected String id;
    @Setter
    protected String name;
    protected List<Item> userBorrowedItems;

    private static int nextId = 1; 1 usage

    public User(String name) { 3 usages SavulescuD
        this.id = String.format("%04d", nextId++);
        this.name = name;
        this.userBorrowedItems = new ArrayList<>();
    }

    @Override SavulescuD
    public int compareTo(@NotNull User o) {
        return name.compareToIgnoreCase(o.name) == 0 ? id.compareTo(o.id) : name.compareToIgnoreCase(o.name);
    }
}
```

```

@Override(callSuper = true) 24 usages  SavulescuD
@Getter
@Setter
@EqualsAndHashCode(callSuper = true)
public class Student extends User {
    public Student(String name) { 13 usages  SavulescuD
        super(name);
    }
}

```

```

@Override(callSuper = true) 14 usages  SavulescuD
@Getter
@Setter
@EqualsAndHashCode(callSuper = true)
public class Teacher extends User {
    public Teacher(String name) { 8 usages  SavulescuD
        super(name);
    }
}

```

```

@Override no usages  SavulescuD
public void reportAll() {
    for (Item item : Library.items) {
        if (item.getStatus().equals(Item.Status.AVAILABLE)) {
            System.out.println(item.getId() + "-" + item.getTitle() + ": Available");
        }

        if (item.getStatus().equals(Item.Status.BORROWED)) {
            System.out.println(item.getId() + "-" + item.getTitle() + ": Borrowed");
        }

        if (item.getStatus().equals(Item.Status.LOST)) {
            System.out.println(item.getId() + "-" + item.getTitle() + ": Lost");
        }
    }
}

/**
 * backs up all the users and books into another file each
 */
public void backup() { no usages  SavulescuD
    Library.loadUsers();
    Library.loadBooks();
}

```

```

@Override(callSuper = true) 4 usages  SavulescuD
@Getter
@Setter
@EqualsAndHashCode(callSuper = true)
public class Admin extends User implements Reportable {
    public Admin(String name) { super(name); }

    @Override no usages  SavulescuD
    public void reportInStore() {
        for (Item item : Library.items) {
            if (item.getStatus().equals(Item.Status.AVAILABLE)) {
                System.out.println(item.getId() + "-" + item.getTitle() + ": Available");
            }
        }
    }

    @Override no usages  SavulescuD
    public void reportBorrowed() {
        for (Item item : Library.items) {
            if (item.getStatus().equals(Item.Status.BORROWED)) {
                System.out.println(item.getId() + "-" + item.getTitle() + ": Borrowed");
            }
        }
    }

    @Override no usages  SavulescuD
    public void reportLost() {
        for (Item item : Library.items) {
            if (item.getStatus().equals(Item.Status.LOST)) {
                System.out.println(item.getId() + "-" + item.getTitle() + ": Borrowed");
            }
        }
    }
}

```

Every user is automatically assigned a unique zero-padded four-digit ID when the user profile is created. Users are also comparable to one another, first by ID then by name, which supports sorting operations.

2.2 Library Items

Items also follow the same pattern. They have an abstract Item class defining the shared structure. Items all have an id, a title and a list of all the borrowed items. Three subclasses handle the specifics. Books are identified by a 13-digit ISBN, author, and genre. DVDs include a director and a duration in minutes. Magazines have an issue number and a publisher.

```
@EqualsAndHashCode 83 usages  SavulescuD
@ToString
@Getter
public abstract class Item {
    protected String id;
    @Setter
    protected String title;
    @Setter
    protected Status status;

    private static int nextId = 1; 1 usage

    public Item(String title, Status status) { 3 usages  SavulescuD
        this.id = String.format("%04d", nextId++);
        this.title = title;
        this.status = status;
    }

    public enum Status { 55 usages  SavulescuD
        AVAILABLE, 38 usages
        BORROWED, 8 usages
        LOST 3 usages
    }

    public static class ItemComparator implements Comparator<Item> { no usages  SavulescuD
        @Override  SavulescuD
        public int compare(@NotNull Item o1, @NotNull Item o2) {
            return o1.getTitle().compareToIgnoreCase(o2.title) == 0 ? o1.getId().compareTo(o2.getId()) : o1.getTitle().compareToIgnoreCase(o2.title);
        }
    }
}
```

Each item has a status using Enum: Available, Borrowed, or Lost. This reflects its current state. Multiple physical copies of the same title can coexist in the system, each with its own unique ID, which allows the borrowing system to track individual copies.

```

@Getter 72 usages SavulescuD
@ToString(callSuper = true)
@EqualsAndHashCode(callSuper = true)
public class Book extends Item {
    private String isbn;
    @Setter
    private String author;
    @Setter
    private String genre;

    public Book(String title, Status status, String isbn, String author, String genre) {
        super(title, status);
        this.isbn = isbn;
        this.author = author;
        this.genre = genre;
    }
}

```

```

@Getter 8 usages SavulescuD
@Setter
@ToString(callSuper = true)
@EqualsAndHashCode(callSuper = true)
public class Dvd extends Item {
    private String director;
    private int duration;

    public Dvd(String title, Status status, String director, int duration) {
        super(title, status);
        this.director = director;
        this.duration = duration;
    }
}

```

```

@Getter 6 usages SavulescuD
@Setter
@ToString(callSuper = true)
@EqualsAndHashCode(callSuper = true)
public class Magazine extends Item {
    private int issueNumber;
    private String publisher;

    public Magazine(String title, Status status, int issueNumber, String publisher) {
        super(title, status);
        this.issueNumber = issueNumber;
        this.publisher = publisher;
    }
}

```

2.3 The Library

The Library class is the central hub of the system. It contains the full list of items and users, and all the key operations: borrowing, returning, searching, loading data from CSV files, and writing backups.

3. Key Features

3.1 Borrowing and Returning

When a user borrows an item, the system checks that the item is available, that the user has not exceeded their limit, and in the case of students, that the item is a book. If any check fails, an `IllegalArgumentException` is thrown with a message. If the borrowing succeeds, the

```

@Getter 125 usages SavulescuD
@Setter
@ToString
@EqualsAndHashCode
public class Library {
    public static List<Item> items; 61 usages
    public static List<User> users; 31 usages

    /**
     * permits the user to borrow items
     *
     * @param item the item to be borrowed
     * @param user the user borrowing the item
     */
    public static void borrowItems(Item item, User user) throws IllegalArgumentException {
        if (!items.contains(item)) {
            throw new IllegalArgumentException("Item is currently unavailable");
        }

        if (user instanceof Student) {
            if (!(item instanceof Book)) {
                throw new IllegalArgumentException("Students can only borrow books");
            }
            if (user.getUserBorrowedItems().size() >= 5) {
                throw new IllegalArgumentException("Student borrow limit reached");
            }
        } else if (user instanceof Teacher) {
            if (user.getUserBorrowedItems().size() >= 10) {
                throw new IllegalArgumentException("Teacher borrow limit reached");
            }
        }

        items.remove(item);
        user.userBorrowedItems.add(item);
        item.setStatus(Item.Status.BORROWED);
    }
}

```

item is removed from the library's available pool, added to the user's borrowed list, and its status is updated. Returning an item reverses this process.

```

/**
 * permits the user to return an item
 *
 * @param item the item to be returned
 * @param user the user returning the item
 */
public static void returnItems(Item item, @NotNull User user) {
    user.userBorrowedItems.remove(item);
    items.add(item);
    item.setStatus(Item.Status.AVAILABLE);
}

```

3.2 Searching

The system provides two independent search implementations, both case-insensitive and both ensuring no duplicated results so multiple copies of the same title only appear once. The stream-based method uses the stream with a filter to ensure that the items containing the keyword only appear once. The recursive method traverses the item list index by index, getting more matches and adding the seen titles through a Set. Both methods search by title and, for books, by author as well.

```

/**
 * enables the user to search for an item using a key word using a stream
 *
 * @param keyword the key word to search for the item
 * @return the results of the search
 */
public static @NotNull @Unmodifiable List<Item> searchStream(@NotNull String keyword) { 6 usages SavulescuD
    String lowerCaseKey = keyword.toLowerCase();
    Set<String> seenTitles = new HashSet<>();

    return items.stream()
        .filter(item -> {
            boolean matchesKeyword = item.getTitle().toLowerCase().contains(lowerCaseKey);
            if (!matchesKeyword && item instanceof Book) {
                matchesKeyword = ((Book) item).getAuthor().toLowerCase().contains(lowerCaseKey);
            }
            return matchesKeyword && seenTitles.add(item.getTitle().toLowerCase());
        })
        .toList();
}

```

```

/**
 * enables the user to search for an item using a key word using recursion
 *
 * @param keyword    the keyword to search for the item
 * @param index      the index to check
 * @param results    the results of the search so far
 * @param seenTitles the titles that have already been checked
 * @return the list of all the items containing that key word
 */
public static List<Item> searchRecursive(String keyword, int index, List<Item> results, Set<String> seenTitles) {
    if (index == items.size()) {
        return results;
    }

    Item currentItem = items.get(index);
    String lowerKeyword = keyword.toLowerCase();
    String title = currentItem.getTitle().toLowerCase();

    boolean matches = title.contains(lowerKeyword);
    if (!matches && currentItem instanceof Book) {
        matches = ((Book) currentItem).getAuthor().toLowerCase().contains(lowerKeyword);
    }

    if (matches && !seenTitles.contains(title)) {
        results.add(currentItem);
        seenTitles.add(title);
    }

    return searchRecursive(keyword, index + 1, results, seenTitles);
}

```

3.3 Reporting

```

public interface Reportable {
    void reportInStore(); no usage
    void reportBorrowed(); no usage
    void reportLost(); no usage
    void reportAll(); no usages
}

```

The Admin class implements a Reportable interface with four methods: reporting borrowed items, in-store items, lost items, and a combined report of everything. This gives administrators a real-time view of the library's state.

3.4 Data Persistence

The system initializes itself from CSV files, one for users, one for items which makes it easy to populate the library without hardcoding the data. Administrators can also trigger a backup at any time, writing the current state back to CSV for future use.

```
/**
 * initializes the users from a user file
 *
 * @param userPath the path for the user file
 */
public static void initializeUsers(String userPath) { no usages  SavulescuD
    File file = new File(userPath);

    try (Scanner scanner = new Scanner(file)) {
        while (scanner.hasNext()) {
            String line = scanner.nextLine();
            String[] elements = line.split(regex: ",");
            /*
             structure for a line should be like so: Occupation,Name
             example -> Student, Name
             */
            String name = elements[1];
            if (elements[0].equalsIgnoreCase(anotherString: "student")) {
                Student student = new Student(name);
                users.add(student);
            } else if (elements[0].equalsIgnoreCase(anotherString: "teacher")) {
                Teacher teacher = new Teacher(name);
                users.add(teacher);
            } else if (elements[0].equalsIgnoreCase(anotherString: "admin")) {
                Admin admin = new Admin(name);
                users.add(admin);
            } else {
                throw new RuntimeException("User indeterminable");
            }
        }
    } catch (FileNotFoundException e) {
        throw new RuntimeException(e);
    }
}
```

```

/**
 * initializes the items from a user file
 *
 * @param itemsPath the path for the item file
 */
public static void initializeItems(String itemsPath) { no usages  SavulescuD
    File file = new File(itemsPath);

    try (Scanner scanner = new Scanner(file)) {
        while (scanner.hasNext()) {
            String line = scanner.nextLine();
            String[] elements = line.split(regex: "%");

            /*
             structure for a line should be like so:
             Book,title,status,isbn,author,genre
             Dvd,title,status,director,duration
             Magazine,title,status,issueNumber,publisher
             example -> Student, Name
             */
            String title = elements[1].toLowerCase();
            Item.Status status;

            if (elements[2].equalsIgnoreCase(anotherString: "borrowed")) {
                status = Item.Status.BORROWED;
            } else if (elements[2].equalsIgnoreCase(anotherString: "available")) {
                status = Item.Status.AVAILABLE;
            } else {
                status = Item.Status.LOST;
            }

            switch (elements[0]) {
                case "book" -> {
                    Book book = new Book(title, status, elements[3], elements[4], elements[5]);
                    items.add(book);
                }

                case "dvd" -> {
                    Dvd dvd = new Dvd(title, status, elements[3], Integer.parseInt(elements[4]));
                    items.add(dvd);
                }

                case "magazine" -> {
                    Magazine magazine = new Magazine(title, status, Integer.parseInt(elements[3]), elements[4]);
                    items.add(magazine);
                }

                default -> throw new RuntimeException("Item indeterminable");
            }
        }
    } catch (FileNotFoundException e) {
        throw new RuntimeException(e);
    }
}

```



```

/**
 * loads users into a csv file
 */
public static void loadUsers() { 2 usages  SavulescuD
    File file = new File(Constants.USERS_CSV_PATH);

    try (FileWriter fileWriter = new FileWriter(file, append: true)) {
        for (User user : users) {
            if (user instanceof Student) {
                fileWriter.write(str: "Student,");
            } else if (user instanceof Teacher) {
                fileWriter.write(str: "Teacher,");
            } else {
                fileWriter.write(str: "Admin,");
            }
            fileWriter.write(str: user.getId() + ",");
            fileWriter.write(str: user.getName() + ",");
            fileWriter.write(str: "Borrowed Items:");
            for (Item item : user.getUserBorrowedItems()) {
                fileWriter.write(str: item.getId() + "-" + item.getTitle());
            }
            fileWriter.write(str: "\n");
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}

```

```

/**
 * loads books into a csv file
 */
public static void loadBooks() { 2 usages  SavulescuD
    File file = new File(Constants.BOOKS_CSV_PATH);

    try (FileWriter fileWriter = new FileWriter(file, append: true)) {
        for (Item item : items) {
            if (item instanceof Book) {
                fileWriter.write(str: item.getId() + ",");
                fileWriter.write(str: item.getTitle() + ",");
                fileWriter.write(str: ((Book) item).getIsbn() + ",");
                fileWriter.write(str: ((Book) item).getAuthor() + ",");
                fileWriter.write(str: ((Book) item).getGenre() + "\n");
            }
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}

```

3.5 Sorting

Both users and items support 2 different sorting strategies. The User class implements a Comparable while the Item class implements a Comparator. Both compare the name at first, and in case they're identical, both compare the name.

3.6 Validation

A dedicated Validation class checks if the structure for the ISBN for the books is respected. Every book's ISBN must be exactly 13 numeric digits. Invalid ISBNs return false.

```

public class Validation { 8 usages  SavulescuD
    /**
     * validates if a book has the correct ISBN structure
     * @param book the book to be checked
     * @return true if the ISBN corresponds to the good structure, false if not
     */
    public static boolean validation(@NotNull Book book) { 7 usages  SavulescuD
        if (book.getIsbn() == null) {
            return false;
        }

        for (Character c : book.getIsbn().toCharArray()) {
            if (!Character.isDigit(c)) {
                return false;
            }
        }

        return book.getIsbn().length() == 13;
    }
}

```

4. Design Decisions

A few choices are worth explaining, as they reflect deliberate trade-offs rather than arbitrary decisions.

Lombok annotations were used to keep domain classes concise. @Getter, @Setter, @ToString, and @EqualsAndHashCode.

The two search methods were kept separate rather than combined. This makes each one easier to read, understand, and test independently.

Equality for both User and Item is based solely on their unique ID. This prevents situations where two objects with identical data but different IDs would be incorrectly treated as duplicates.

5. Testing

Unit tests were written using JUnit, covering all major components.

All testable methods from the Library class were tested in the LibraryTest class. Each method was tested multiple times with different cases to ensure that the methods were working properly. The same was applied to test the Validation class method in the ValidationTest class.

```
public class LibraryTest {
    // testing borrowItems method
    @Test
    @DisplayName("Student can borrow a book successfully")
    public void borrowItemsTest1() {
        Library library = new Library();
        library.items = new ArrayList<>();
        library.users = new ArrayList<>();

        Student student = new Student(name: "Alice");
        Book book = new Book(title: "Harry Potter", Item.Status.AVAILABLE, isbn: "9780747532743", author: "J.K. Rowling", genre: "Fantasy");
        library.items.add(book);

        library.borrowItems(book, student);

        assertEquals(Item.Status.BORROWED, book.getStatus());
    }

    @Test
    @DisplayName("Student cannot borrow a DVD")
    public void borrowItemsTest2() {
        library.items = new ArrayList<>();
        library.users = new ArrayList<>();
        Library library = new Library();

        Student student = new Student(name: "Alice");
        Dvd dvd = new Dvd(title: "Inception", Item.Status.AVAILABLE, director: "Christopher Nolan", duration: 140);
        library.items.add(dvd);

        assertThrows(IllegalArgumentException.class, () -> library.borrowItems(dvd, student));
    }
}
```

```
public class ValidationTest {
    @Test
    @DisplayName("Valid 13-digit ISBN returns true")
    public void validationTest1() {
        Book book = new Book(title: "Clean Code", Item.Status.AVAILABLE, isbn: "9780132350884", author: "Robert Martin", genre: "Tech");
        boolean expected = true;
        boolean actual = Validation.validation(book);
        Assertions.assertEquals(expected, actual);
    }

    @Test
    @DisplayName("ISBN with less than 13 digits returns false")
    public void validationTest2() {
        Book book = new Book(title: "Clean Code", Item.Status.AVAILABLE, isbn: "978013235", author: "Robert Martin", genre: "Tech");
        boolean expected = false;
        boolean actual = Validation.validation(book);
        Assertions.assertEquals(expected, actual);
    }

    @Test
    @DisplayName("ISBN with more than 13 digits returns false")
    public void validationTest3() {
        Book book = new Book(title: "Clean Code", Item.Status.AVAILABLE, isbn: "97801323508841234", author: "Robert Martin", genre: "Tech");
        boolean expected = false;
        boolean actual = Validation.validation(book);
        Assertions.assertEquals(expected, actual);
    }
}
```

6. Conclusion

The Library Management System should meet all the requirements outlined in the project instructions. It simulates a realistic library domain using object-oriented design.

This project taught me that how you set things up at the beginning really matters. Small decisions, like where to put the how to generate IDs, ended up affecting a lot of other parts of the code later on. It was harder to change things once everything was connected, which made me realize why planning ahead is important.

Testing was also more useful than I expected. I caught several bugs that looked fine at first but were actually broken, like the search method that was returning empty results every time before getting fixed. Without the tests I probably would have missed that.

Overall , this was a good way to practice the concepts from class on something big enough to actually feel the difference between writing code that just works and writing good code that is structured.